Real-Time Payment Fraud Detection

Business Analytics Group Project — Modules 1–8

[Team member names]

Business Analytics

Agenda

1. Business problem & KPIs *(M1)*
2. Two data sources *(M1)*
3. **The leakage trap** *(design)*
4. Key EDA findings *(M2)*
5. Data cleaning decisions *(M3)*
6. Feature engineering *(M4)*
7. Model results & cost threshold *(M5)*
8. **What our numbers really mean**
9. Deploy: API + review queue *(M6)*
10. Monitoring + retraining *(M7)*
11. Risk policy *(M8)* + conclusion

The one thing to remember

Our best model reaches PR-AUC 0.997 — and the interesting part is **why that number is not as impressive as it looks**.

1. The business problem

The question we must answer

"Which transactions are likely fraudulent, and how should the platform balance blocking fraud against approving legitimate orders without unnecessary friction?"

Fraud is costly

- Only **0.129%** of transactions
- But $8.2\times$ **larger** on average
- Total exposure: ≈ 12.1 **billion**

But blocking is costly too

- Every false alarm = a rejected customer
- Friction $\rightarrow$ abandoned carts, churn
- Goal is *not* "catch everything"

$\Rightarrow$ Find the operating point that minimizes **total business cost**

1. KPIs we track

KPI	What it measures	Baseline
Fraud rate	Risk exposure & imbalance	0.129%
False-positive rate	Customer friction	to minimize
Financial loss avoided	Business value	target

Why accuracy is useless here

A model that predicts "never fraud" achieves **99.87% accuracy** and catches **zero** fraud.
$\Rightarrow$ We use **PR-AUC**, Recall, F_1 — never accuracy.

2. Two data sources

Source 1 — PaySim (Kaggle)

- 6,362,620 transactions
- 743 hourly steps ($\approx$ 31 days)
- Balances before/after, type, amount
- Ground-truth `isFraud`

Source 2 — Synthetic (Faker)

- **10 contextual features**
- Account age, home country, device
- New-device flag, address mismatch
- Failed attempts, IP distance
- Fully documented in a data dictionary

Generation was label-conditional — on purpose

e.g. `is_new_device`: $p = 0.55$ if fraud, 0.08 if legitimate
`failed_payment_attempts`: Poisson $\lambda = 2.2$ vs 0.12

3. The trap we found in our own data

Our synthetic features separate the classes beautifully:

Feature	Legitimate	Fraud	Ratio
`account_age_days`	500.1	44.1	0.09×
`is_new_device`	0.080	0.551	6.9×
`failed_payment_attempts`	0.120	2.249	18.7×
`ip_billing_mismatch`	0.020	0.499	25.0×

But we generated them *from* the label

Their predictive power is **by design**, not discovered. Using them naively would be **label leakage** — we would be grading our own homework.

Our solution: build **two** feature spaces and compare them everywhere.

3. The Base / Full design

Base — 11 features (no leakage)

Real PaySim data only

- amount, 4 balance columns
- `errorBalance*` (engineered)
- `type_TRANSFER`
- velocity, time-since-last

No leakage — honest difficulty

Full — 18 features (leaky)

Base + 7 synthetic risk signals

- account age, device flag
- address / IP mismatch
- failed attempts, IP distance

Leaky — optimistic upper bound

Every algorithm is trained on **both**.

The gap between them *measures* the leakage instead of hiding it.

4. EDA: two findings that shaped everything

(a) Fraud lives in only 2 of 5 types

Type	Fraud	Rate
CASH_OUT	4,116	0.184%
TRANSFER	4,097	**0.769%**
CASH_IN	0	0%
PAYMENT	0	0%
DEBIT	0	0%

$\Rightarrow$ Filter to 2 types: positive rate
0.129% $\rightarrow$ 0.296% at zero information cost

(b) Fraud breaks the accounting identity

$$\texttt{errBalOrig} = \texttt{newBal} + \texttt{amt} - \texttt{oldBal}$$

Fraud *drains* the origin account, leaving an arithmetic footprint.

Naive rule "account emptied":
fraud rate only 0.223% (1.7$\times$)

The *error* formulation is far sharper
$\rightarrow$ becomes our top feature

5. Cleaning: the decision we are proudest of

What we found

- 0 missing values, 0 duplicates
- 16 invalid rows removed
- IQR rule flags **338,077** outliers (5.31%)

We kept them — here is why

Fraud rate *inside* the outlier group:

1.14% vs 0.129% overall

$= 8.8\times$ denser in fraud

The textbook says: remove outliers

Clipping the tail would have deleted our rare positives.

In fraud detection, a large amount is **signal**, not noise.

Every cleaning step logged rows *and* fraud removed — verified with assertions.

6. Feature engineering & imbalance

Handling 1:337 imbalance

- Class weights in *all* 3 models
- LogReg: `balanced`
- XGBoost: `scale_pos_weight` ≈ 337
- RF: `balanced_subsample`

All impose the *same* 337:1 penalty $\Rightarrow$ fair comparison.

Feature selection (mutual information)

- MI ranks `errorBalanceOrig` only 5th...
- ...yet it is the #1 feature in the models
- MI is *univariate*, blind to interactions

$\Rightarrow$ We kept all features.

Blind pruning would have deleted our best predictor.

An honest negative result

Required velocity features (`txn_count_acct`, `time_since_last_txn`) are **degenerate on PaySim** — accounts almost never repeat (mean count = 1.00). Built correctly, but carry **zero** importance.

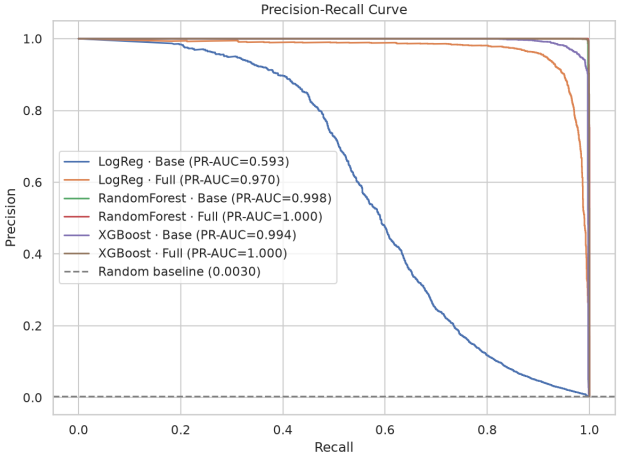
7. Results: 3 algorithms $\times$ 2 feature spaces

Model	PR-AUC	Recall	Precision	FP	FN
LogReg · Base	0.5932	0.9081	0.0433	**49,287**	226
LogReg · Full	0.9704	0.9947	0.3348	4,860	13
RF · Base	**0.9976**	0.9967	0.9996	**1**	8
RF · Full	0.9997	0.9963	1.0000	0	9
XGBoost · Base	0.9944	0.9931	0.9222	206	17
XGBoost · Full	0.9996	0.9984	0.9923	19	4

Random Forest Base — best *honest* model:
1 false positive out of 828,659
using **no synthetic data at all**

LogReg Base is unusable:
22 false alarms per fraud caught
(a hyperplane cannot express the balance interaction)

7. Precision–Recall curves



The linear baseline degrades steadily; the four tree curves hug the corner.

8. Measuring the leakage we warned about

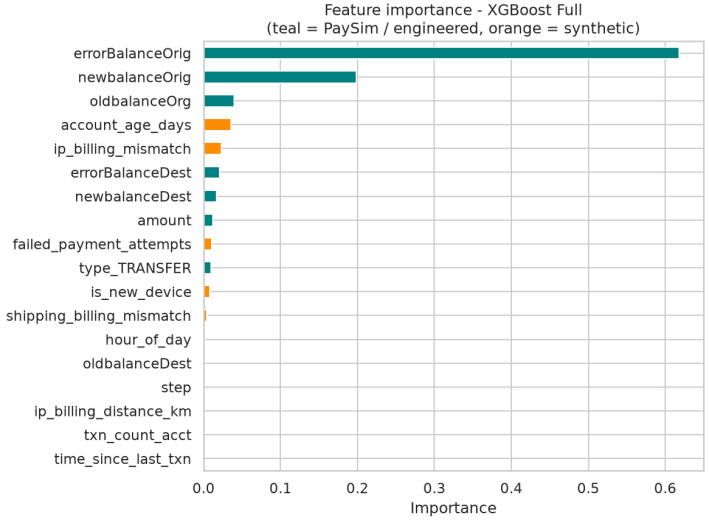
Algorithm	Base	Full	Uplift from synthetic data
Logistic Regression	0.5932	0.9704	+0.3772
Random Forest	0.9976	0.9997	+0.0021
XGBoost	0.9944	0.9996	+0.0052

What this tells us

- The leakage is **real** — it lifts the weak model by +0.38
- But it is **not load-bearing** — tree models gain almost nothing
- Our best honest model (RF Base) **needs none of it**

Conclusion: our headline result does not rest on leaked information.

8. Where the signal actually comes from



Teal = real PaySim
Orange = synthetic

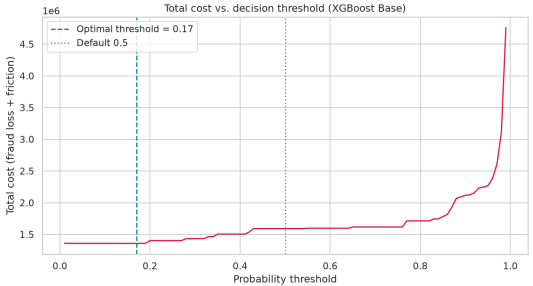
- `errorBalanceOrig` alone = **53.7%**
- Synthetic total $\approx 28\%$
- Velocity features = **0.000**

The tallest bar is a *real* feature.

9. From probability to a business decision

The default threshold 0.5 has *no business meaning*. We convert the trade-off into money:

$$\mathrm{Cost}(t) = \underbrace{\sum_{\text{missed fraud}} \texttt{amount}_i}_{\text{money lost}} + \underbrace{C_{FP} \times \#\text{false alarms}}_{\text{friction, } C_{FP}=10}$$



Optimal $t^* = 0.17$, not 0.5 $\Rightarrow$ total cost -14.7%

9. The business outcome

Indicator (test set)	Value
Fraud value at risk	3,615,247,576
Fraud value stopped	3,613,891,825 **(99.96%)**
Fraud value leaked	1,355,751
False-positive rate	0.032% (265 transactions)

Compare with the incumbent rule-based system

PaySim's built-in `isFlaggedFraud` flag catches **16** frauds
Our model catches **2,451** of 2,459

The cost curve is *flat* below $t \approx 0.25$: the optimum is a broad plateau, so the result is robust to our C_{FP} assumption.

10. What our numbers really mean

PR-AUC 0.997 with 1 false positive is *not* a brag

In real fraud detection such numbers would be implausible. The explanation is **the data, not the model**.

- PaySim scripts fraud as "empty the account" $\Rightarrow$ an **almost deterministic** arithmetic footprint
- `errorBalanceOrig` captures it directly (53.7% importance)
- Evidence: the *linear* model fails (0.59) while both tree models exceed 0.99 $\Rightarrow$ one nonlinear, near-deterministic rule
- Our synthetic features add leakage on top

Real-world performance would be substantially lower.
We report this as a **limitation**, not a headline.

11. Deployment: real-time API + review queue

Served model: XGBoost Base (8 features)

- `POST /predict` — score one txn in real time
- `GET /health` — model metadata
- `featurize()` rebuilds `errorBalance*` and `type_TRANSFER` on the server

Analyst review queue (Streamlit)

- ranks a batch high → low risk
- per-txn approve / block + action log

Free-tier deploy (Hugging Face Spaces), one-command push.

FastAPI + Streamlit sharing one scoring core.

12. Monitoring: drift, performance, retraining triggers

- **Data drift** — PSI + Kolmogorov–Smirnov on the 8 served features
- **Performance over time** — rolling precision / recall / PR-AUC by time window
- **Retraining triggers** (fire on *any*):
  - max PSI > 0.2 or ≥ 2 features drifted
  - rolling recall < 0.90 or precision < 0.20
  - rolling PR-AUC > 0.1 below reference

Own implementation (`monitoring_core.py`) $\rightarrow$ HTML dashboard; a stress test confirms the alerts fire.

13. Risk policy: from a score to a business action

Score band	Tier	Action
$p < t^*$	auto-approve	clear immediately
$t^* \leq p < b$	manual-review	analyst queue
$p \geq b$	auto-block	block immediately

- $t^* \approx 0.09$ (deployed model, cost-optimal); b = smallest score with precision ≥ 0.99
- Auto-block only when **near-certain**; grey zone $\rightarrow$ human review
- Escalation signal = the `errorBalanceOrig` balance mismatch

Conclusion

What we delivered

- End-to-end pipeline, Modules 1–8
- 2 data sources + full data dictionary
- Documented cleaning with audit log
- 3 algorithms $\times$ 2 feature spaces
- Cost threshold + deploy + monitoring + risk policy

Our recommendation

- **Random Forest Base** — best honest model
- **XGBoost Base** — deploy (compact, fast, leak-free)
- Operate at $t^* = 0.17$

The contribution we value most

We turned a hidden **label-leakage flaw** into a **measured quantity**, and we can explain *exactly* why our scores are high.

Next: methodological hardening — temporal split, cross-validation, C_{FP} from real costs, validation on genuine transaction data

Thank you

Questions?

https://github.com/Trungdn4-458311/FraudDetectionBE

Backup: why not SMOTE?

- The brief allows SMOTE *or* undersampling *or* class weights
- We chose **class weights** because:
  - It does not fabricate synthetic positives in a dataset that *already* contains synthetic components
  - It leaves the test distribution untouched
  - It is far cheaper on 1.9M training rows
- Applied consistently across all three algorithms at the same 337:1 ratio

Backup: why does RF beat XGBoost at $t = 0.5$?

- Ranking gap is *small*: PR-AUC 0.9976 vs 0.9944
- Precision gap is *large*: 0.9996 vs 0.9222 (1 vs 206 FP)
- $\Rightarrow$ the difference is **calibration**, not ranking ability
- XGBoost multiplies positive gradients by 337, inflating probabilities toward the positive class; many negatives land above 0.5 while still ranked below true positives
- This is exactly why we tune the threshold instead of trusting 0.5

Backup: limitations

1. **Simulated data** — PaySim fraud is more regular than reality
2. **Synthetic features are label-derived** — strength is our parameter choice, not evidence
3. **Velocity features degenerate** — accounts rarely recur
4. **Random, not temporal split** — production faces time-based generalization
5. $C_{FP} = 10$ **assumed**, not measured
6. **Single split**, no cross-validation variance reported